# Architektury i technologie systemów internetowych (2026L)

## Dokumentacja aplikacji internetowej do obsługi cyfrowego archiwum społecznościowego

Patryk Grochowicz

305779

1. Wstęp ..... 2
1.1 Link do repozytorium GitHub ..... 2
2. Uruchamianie lokalne aplikacji ..... 2
2.1 Wymagane narzędzia ..... 2
2.2 Uruchamianie lokalnie MiniO ..... 2
2.3 Uruchomienie lokalne bazy danych ..... 3
2.4 Uruchamianie lokalnie aplikacji Backendowej (python) ..... 4
2.5 Uruchomienie lokalnie aplikacji frontendowej (React) ..... 6
3. Architektura ..... 7
3.1 Architektura systemu ..... 7
3.2 Opis Struktury Bazy danych ..... 8
3.3 Uzasadnienie struktury bazy danych ..... 10
3.4 Struktura plików w aplikacji backendowej ..... 10
3.5 Uzasadnienie przyjętego rozwiązania aplikacja backendowa ..... 11
3.6 Struktura plików w aplikacji frontendowej ..... 12
3.7 Architektura informacji ..... 13
3.7.1 Nawigacja ..... 13
3.7.2 Filtry ..... 14
3.7.3 Metadane ..... 15
3.7.4 Uzasadnienie Architektury Informacji ..... 16
3.8 Uzasadnienie przyjętego rozwiązania aplikacja frontendowa ..... 16
3.9 Bezpieczeństwo ..... 17
3.10 Autoryzacja ..... 17
4. Dostępność w aplikacji ..... 18
5. Endpointy ..... 19

# 1. Wstęp

Aplikacja omawiana w dokumentacji powstała jako projekt na przedmiot Architektury i technologie systemów internetowych (2026L). Celem aplikacji jest wspieranie lokalnych mieszkańców i pasjonatów historii w gromadzeniu i udostępnianiu zdjęć przedstawiających historię i ewolucję danego obszaru.

## 1.1 Link do repozytorium GitHub

https://github.com/PGrochowicz/Architektury_i_technologie_system-w_internetowych_2026L

# 2. Uruchamianie lokalne aplikacji

## 2.1 Wymagane narzędzia

Do lokalnego uruchomienia aplikacji zalecane są następujące narzędzia w podanych wersjach:

1. Python w wersji 3.13.9 (https://www.python.org/downloads/release/python-3139/)
2. Node w wersji 24.11.0 (https://nodejs.org/en/download)
3. Microsoft SQL Server 2022 Developer Edition
4. SQL Server Management Studio w wersji 20.2.30.0
5. Visual Studio Code
6. Docker Desktop w wersji 4.34.3
7. Microsoft C++ Build Tools

## 2.2 Uruchamianie lokalnie MiniO

Jako że bazy relacyjne nie są zoptymalizowane pod pliki binarne, został wykorzystany MinIO który udostępnia publiczny URL do pliku. Do wystawienia go lokalnie został przygotowany plik ‘docker-compose.yml’. W celu uruchomienia go lokalnie należy wykonać następujące kroki:

1. Uruchom terminal np. PowerShell lub Git Bash.
2. Przejdź do folderu projektu przy pomocy np. Komendy
   cd Architektury_i_technologie_system-w_internetowych_2026L
3. Zbuduj kontener dockerowy przy pomocy komendy: docker compose up

Po poprawnym wykonaniu powyższych kroków w konsoli powinien być widoczny rezultat w następujący sposób:

```
$ docker compose up
[+] Running 3/3
 ✓ Network zai_projekt_default           Created
 ✓ Container zai_projekt-minio-1         Created
 ✓ Container zai_projekt-createbucket-1  Created
Attaching to createbucket-1, minio-1
minio-1         | INFO: Formatting 1st pool, 1 set(s), 1 drives per set.
minio-1         | INFO: WARNING: Host local has more than 0 drives of set. A host failure will result in data becoming unavailable.
minio-1         | MinIO Object Storage Server
minio-1         | Copyright: 2015-2026 MinIO, Inc.
minio-1         | License: GNU AGPLv3 - https://www.gnu.org/licenses/agpl-3.0.html
minio-1         | Version: RELEASE.2025-09-07T16-13-09Z (go1.24.6 linux/amd64)
minio-1         |
minio-1         | API: http://172.18.0.2:9000  http://127.0.0.1:9000
minio-1         | WebUI: http://172.18.0.2:9001 http://127.0.0.1:9001
minio-1         |
minio-1         | Docs: https://docs.min.io
createbucket-1  | Added `local` successfully.
createbucket-1  | Bucket created successfully `local/photos`.
createbucket-1  | Access permission for `local/photos` is set to `download`
createbucket-1 exited with code 0
```

Rysunek 1: Poprawny rezultat przykład Git Bash

Dodatkowo do tego w programie Docker Desktop w zakładce Containers powinny być widoczne dwa kontenery:

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | minio-1 | d441a0e7f367 | minio/mini | 9000:9000 90... | 0% | 8 mir | | |
| | createbucket-1 | 02ade4beb540 | minio/mc:la | - | 0% | 8 mir | | |

Rysunek 2: Kontenery Docker Desktop

Kontener minio-1 reprezentuje aplikacje, która jest wykorzystywana dalej w systemie i do poprawnego działania całego systemu musi być uruchomiony. Natomiast kontener createbucket-1 reprezentuje kontener, który podczas budowania wywołał komendy wymagane do tworzenia bucketa w którym przechowywane są zdjęcia, nie jest on niezbędny do działania systemu.

## 2.3 Uruchomienie lokalne bazy danych

Do wystawienia lokalnie bazy danych w prosty sposób został przygotowany skrypt ‘SQL_DigitalArchive.sql’, który tworzy instancję bazy danych SQL_DigitalArchive oraz zasila ją wszystkimi województwami, rolami użytkowników oraz zaproponowanymi tagami. Dane umożliwiają szybkie rozpoczęcie pracy z aplikacją oraz testowanie jej funkcjonalności.

Aby uruchomić skrypt należy wykonać następujące kroki:

1. Uruchom SQL Server Managment Studio.
2. Wprowadź poprawne dane logowania do instancji SQL Servera na którym ma być wystawiona baza danych.
3. Otwórz plik ‘SQL_DigitalArchive.sql’ w SQL Server Managment Studio np. Poprzez dwukrotne kliknięcie w niego.

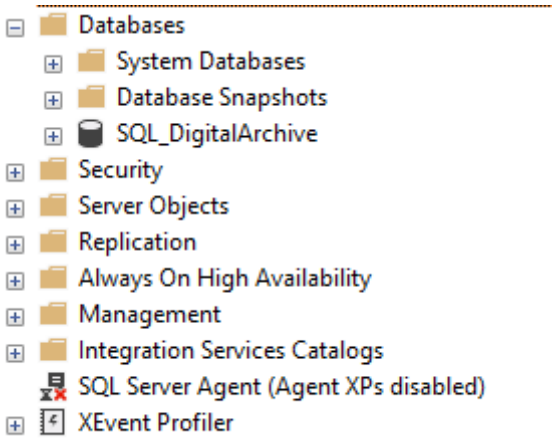
4. Uruchom skrypt przy pomocy przycisku Execute znajdującym się w górnym menu programu

Pozytywny wynik działania skryptu powinien prezentować się następująco:

```
(16 wierszy, które zostały naruszone)

(7 wierszy, które zostały naruszone)

(2 wierszy, które zostały naruszone)

Czas ukończenia: 2026-04-03T12:23:53.3038204+02:00
```

Rysunek 3: Pozytywny rezultat działania skryptu SQL_DigitalArchive.sql

Jeżeli uruchomienie skryptu zakończyło się pozytywnym rezultatem powinna się pojawić baza danych w panelu bocznym aplikacji.




Rysunek 4: Instancja SQL_DigitalArchive

## 2.4 Uruchamianie lokalnie aplikacji Backendowej (python)

Kod aplikacji backendowej znajduje się w folderze ‘packages/backend’. W celu uruchomienia aplikacji backendowej lokalnie należy wykonać następujące kroki:

1. Uruchom terminal np. PowerShell lub Git Bash.
2. Przejdź do folderu projektu przy pomocy np. komendy:
   cd Architektury_i_technologie_system-w_internetowych_2026L/packages/backend
3. Utwórz wirtualne środowisko przy pomocy komendy: python -m venv venv
4. Aktywuj wirtualne środowisko przy pomocy komendy:
   - Git Bash: source venv/Scripts/activate
   - PowerShell: venv/Scripts/activate
5. Zainstaluj wymagane biblioteki przy pomocy komendy: pip install -r requirements.txt, plik ten zawiera listę paczek oraz ich wersje.

6. Uzupełnij plik '.env' znajdujący się w katalogu projektu. Należy je uzupełnić inaczej aplikacja nie będzie działała, ponieważ nie ma fallbacku do wartości domyślnych. W pliku powinny znaleźć się następujące zmienne:

- SQL_CONNECTION_STRING – connection string do lokalnej bazy SQL,
- MINIO_ENDPOINT - url pod którym stoi minio np. http://localhost:9000
- MINIO_ACCESS_KEY – klucz dostępu do minio w pliku dockerowym jest on ustawiony na "minio" i tą wartość należy wkleić chyba że została zmieniona
- MINIO_SECRET_KEY – sekret do minio w pliku dockerowym jest on ustawiony na "minio123" i tą wartość należy wkleić chyba że została zmieniona
- MINIO_BUCKET – nazwa bucketa na zdjęcia w pliku dockerowym jest on ustawiony na "photos" i tą wartość należy wkleić chyba że została zmieniona
- MINIO_PUBLIC_URL – url publiczny do minio powinien on być w formie "MINIO_ENDPOINT/MINIO_BUCKET" np. http://localhost:9000/photos
- GOOGLE_CLIENT_ID – identyfikator klienta od google instrukcja uzyskania dostępna jest pod linkiem https://developers.google.com/identity/oauth2/web/guides/get-google-api-clientid?hl=pl .
  Należy też wspomnieć że w wartościach w konfiguracji klienta google należy podać:
  - Autoryzowane źródła JavaScriptu – url pod którym stoi backend np. http://localhost:8000
  - Autoryzowane identyfikatory URI przekierowania – url pod którym stoi backend zakończony endpointem callbacku np. http://localhost:8000/api/auth/google/callback
- GOOGLE_CLIENT_SECRET – sekret klienta wygenerowanego w instrukcji wyżej można go utworzyć w sekcji "Tajne klucze klienta" w panelu klienta
- BACKEND_URL – url pod którym stoi backend np. http://localhost:8000
- FRONTEND_URL – url pod którym stoi frontend np. http://localhost:5173
- JWT_SECRET - wartość sekretu jwt
- ALLOWED_ORIGINS - lista adresów frontendowych oddzielonych przecinkami, np. http://localhost:5173,http://127.0.0.1:5173

```
SQL_CONNECTION_STRING=DRIVER={ODBC Driver 17 for SQL Server};SERVER={nazwa serwera};DATABASE={nazwa bazy danych};Trusted_Connection=yes;Connection Timeout=10;

MINIO_ENDPOINT={url pod którym stoi minio domyślnie powinno być http://localhost:9000}
MINIO_ACCESS_KEY={access key w pliku docker compose ustawiony domyślnie na minio}
MINIO_SECRET_KEY={secret key w pliku docker compose ustawiony domyślnie na minio123}
MINIO_BUCKET={nazwa bucketa w pliku docker compose ustawiony domyślnie na photos}
MINIO_PUBLIC_URL={url publiczny domyślnie powinien być http://localhost:9000/photos}

GOOGLE_CLIENT_ID={google client ID}
GOOGLE_CLIENT_SECRET={google client secret}
BACKEND_URL={url pod którym stoi aplikacja backendowa}
FRONTEND_URL={url pod którym stoi aplikacja frontendowa}
JWT_SECRET={wartość sekretu jwt}
ALLOWED_ORIGINS={Adresy na których stoi aplikacja frontendowa}
```

Rysunek 5: Zawartość pliku .env

Po poprawnym wykonaniu powyższych kroków aplikacja backendowa będzie gotowa do uruchomienia w środowisku lokalnym przy pomocy komendy: uvicorn main:app --reload --port 8000. Co powinno dać następujący wynik.

```
(.venv) PS C:\source\ZAI_Projekt\packages\backend> py -m uvicorn main:app --reload --port 8000
INFO:     Will watch for changes in these directories: ['C:\\source\\ZAI_Projekt\\packages\\backend']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [19344] using WatchFiles
INFO:     Started server process [20804]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

Rysunek 6: Odpowiedź poprawnie uruchomionej aplikacji

## 2.5 Uruchomienie lokalnie aplikacji frontendowej (React)

Kod aplikacji frontendowej znajduje się w folderze ‘packages/frontend’. W celu uruchomienia aplikacji frontendowej lokalnie należy wykonać następujące kroki:

1. Uruchom terminal np. PowerShell lub Git Bash.
2. Przejdź do folderu projektu przy pomocy komendy:
   cd Architektury_i_technologie_system-w_internetowych_2026L/packages/frontend
3. Zainstaluj wymagane paczki przy pomocy komendy: npm install
4. Uzupełnij plik ‘.env’ znajdujący się w katalogu projektu. W pliku powinny znaleźć się następujące zmienne:
   - REACT_APP_API_URL – adres url api z którego aplikacja frontednowa będzie korzystać
   - REACT_APP_MINIO_URL – adres url pod którym stoi bucket minio z zdjęciami np. http://localhost:9000/photos

Po poprawnym wykonaniu powyższych kroków aplikacja frontendowa będzie gotowa do uruchomienia w środowisku lokalnym przy pomocy komendy: npm run dev. Jeżeli wszystkie kroki w rozdziale drugim zostały poprawnie wykonane powinna pojawić się inicjalna strona aplikacji wraz z danymi pobranymi z bazy danych.

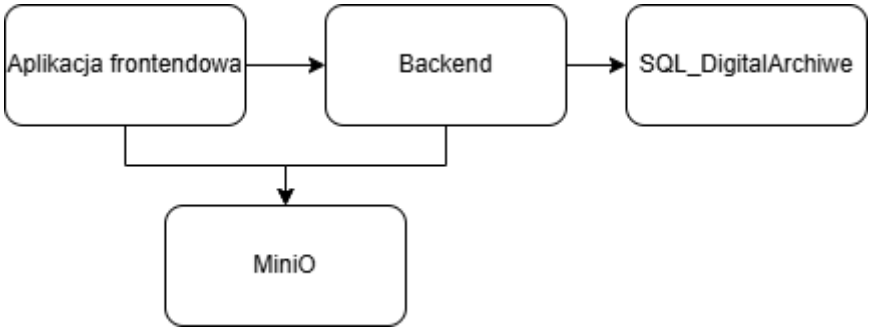
Rysunek 7: Inicjalna strona aplikacji

# 3. Architektura

## 3.1 Architektura systemu

W skład systemu wchodzą cztery aplikacje przedstawione na diagramie:

- Frontend – aplikacja SPA napisana z wykorzystaniem frameworka React w języku TypeScript. Do budowania komponentów wykorzystana została w niej biblioteka MUI. Komunikuje się po http z BE oraz pobiera zdjęcia za pomocą linku url z MiniO które są wyświetlane.
- Backend – aplikacja napisana w języku Python z wykorzystaniem FastApi, pyodbc. Udostępnia Rest Api dla aplikacji przy pomocy którego można się komunikować. Odpowiedzialna jest też za integracje z logowaniem Google oraz obsługą zdjęć w buckecie udostępnionym przez MiniO.
- SQL_DigitalArchive – relacyjna baza danych przechowująca wszystkie niezbędne informacje wymagane do działania systemu.
- MiniO - jako że bazy relacyjne nie są zoptymalizowane pod pliki binarne oraz standardem jest trzymanie zdjęć w bucketach wykorzystane zostało najpopularniejsze rozwiązanie open-source typu object storage MiniO, które udostępnia bucket w którym umieszczane są zdjęcia.




Rysunek 8: Diagram prezentujący aplikacje

## 3.2 Opis Struktury Bazy danych

Baza danych aplikacji zawiera osiem tabeli przechowujących informacje wymagane do poprawnego działania aplikacji:

1. Users – tabela zawierająca informacje o kontach użytkowników posiadająca pola:

- Id – pole o typie int zawierające klucz główny które jest autoinkrementowalne
- GoogleSub – pole o typie nvarchar(100) przechowujące unikalny identyfikator użytkownika google
- Email – pole o typie nvarchar(255) przechowujące email użykownika
- DisplayName – pole o typie nvarchar(255) przechowujące nazwę wyświetlaną dla użytkownika
- IsActive – pole o typie bit przechowujące informacje czy użytkownik jest aktywny
- CreatedAt – pole o typie datetime2(7) przechowujące informacje o tym, kiedy użytkownik po raz pierwszy się zalogował
- LastLoginAt - pole o typie datetime2(7) przechowujące informacje o tym, kiedy użytkownik ostatnio się logował

2. Cities - tabela zawierająca informacje o miastach posiadająca pola:

- Id – pole o typie int zawierające klucz główny które jest autoinkrementowalne
- Name - pole o typie nvarchar(150) przechowujące nazwe miasta
- VoivodeshipId – pole przedstawiające klucz obcy do tabeli województw (Voivodeship) przypisujące miasto do województwa

3. Voivodeships - tabela zawierająca informacje o województwach posiadająca pola:

- Id – pole o typie int zawierające klucz główny które jest autoinkrementowalne
- Name - pole o typie nvarchar(100) przechowujące nazwę województwa

4. Roles - tabela zawierająca informacje o rolach posiadająca pola:

- Id – pole o typie int zawierające klucz główny które jest autoinkrementowalne
- Name - pole o typie nvarchar(50) przechowujące nazwe roli

5. UserRoles - tabela zawierająca informacje o rolach przypisanych do użytkownika posiadająca pola:

- UserId – pole o typie int zawierające id użytkownika
- RoleId - pole o typie int zawierające id roli

6. Tags - tabela zawierająca informacje o tagach posiadająca pola:

- Id – pole o typie int zawierające klucz główny które jest autoinkrementowalne
- Name - pole o typie nvarchar(100) przechowujące nazwę tagu

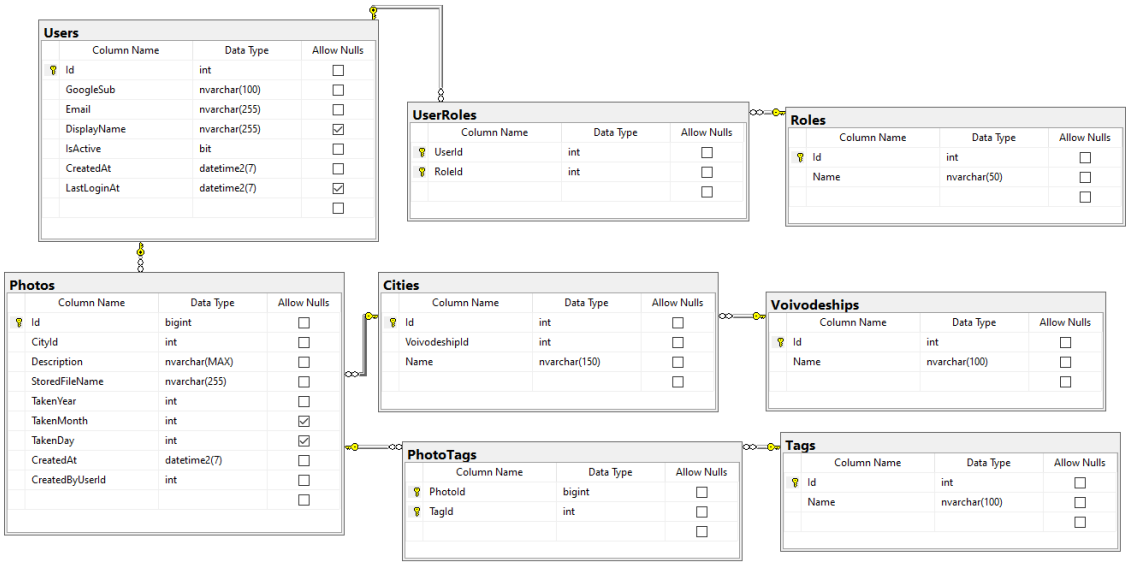
7. PhotoTags - tabela zawierająca informacje o tagach przypisanych do zdjęcia posiadająca pola:

- PhotoId – pole o typie bigint zawierające id zdjęcia
- TagId - pole o typie int zawierające id tagu

8. Photos - tabela zawierająca informacje o zdjęciach posiadająca pola:

- Id – pole o typie int zawierające klucz główny które jest autoinkrementowalne
- CityId– pole przedstawiające klucz obcy do tabeli miast (Cities) przypisujące zdjęcie do miasta
- Description – pole o typie nvarchar(MAX) przechowujące opis zdjęcia
- StoredFileName– pole o typie nvarchar(255) przechowujące nazwe zdjęcia w MiniO
- TakenYear– pole o typie int przechowujące rok, w którym zdjęcie zostało zrobione
- TakenMonth– pole nullowalne o typie int przechowujące miesiąc, w którym zdjęcie zostało zrobione
- TakenDay - pole nullowalne o typie int przechowujące dzień, w którym zdjęcie zostało zrobione
- CreatedAt – pole o typie datetime2(7) przechowujące informacje o tym, kiedy zdjęcie zostało dodane
- CreatedByUserId - pole przedstawiające klucz obcy do tabeli Użytkowników (Users) przypisująće użytkownika, który utworzył zdjęcie

Poniżej znajduję się diagram ERD który przedstawia zależności między tabelami:




Rysunek 9: Diagram ERD

## 3.3 Uzasadnienie struktury bazy danych

Przedstawiona struktura bazy danych została przyjęta z następujących powodów:

- Hierarchia Voivodeship -> City odzwierciedla naturalny podział lokalizacji na mapach Polski. Najwyżej znajduje się województwo, a w województwie miasta. Taki podział ułatwia nawigację oraz filtrowanie.
- Tagi reprezentowane w relacji wiele do wielu pozwalają za pomocą predefiniowanych przez administratora tagów dodać kategorię do zdjęcia. Dodatkowo pozwalają zawężać wyszukiwane treści na bazie predefiniowanych wartości eliminując występowanie literówek.
- Użytkownicy logują się przy pomocy Google przez co nie trzeba zarządzać loginami i hasłami. GoogleSub to unikalny identyfikator dostarczany przez Google. Pole isActive pozwala zbanować użytkownika bez usuwania jego zdjęć oraz konta przez co użytkownik może odwołać się od bana nie musząc znowu uzupełniać swojego archiwum.
- Nazwy zdjęć (storedFileName) są przechowywane docelowo jako UUID co pozwala zapobiegać kolizją w nazewnictwie plików.
- Elastyczna data składająca się z pól TakenYear, TakenMonth i TakenDay pozwala definiować datę utworzenia zdjęcia z dowolną dokładnością. Wymagane jest zawsze uzupełnione pole TakenYear.
- Tabela zdjęć przechowuje tylko informacje o zdjęciu, natomiast samo zdjęcie znajduje się w MiniO.

## 3.4 Struktura plików w aplikacji backendowej

Aplikacja backendowa w celu zapewnienia czytelności kodu została podzielona na następujące pliki:

1. main.py – główny plik odpowiadający za rejestracje routerów oraz logikę CORS
2. .env – zmienne środowiskowe wykorzystywane w kodzie
3. routes – folder zawierający kod odpowiadający za logikę endpointów znajdują się w nim następujące foldery i pliki:

- auth/
    - auth.py - plik zawierający logikę logowania z OAuth2, Google, Jwt oraz obsługa ciasteczka autoryzacyjnego jak i zarówno endpointy odpowiedzialne za zarządzanie użytkownikami i pobieranie danych o użytkowniku
- photos/
    - public.py - plik zawierający endpoint do pobierania zdjęć jest klasyfikowany jako public api oraz widoczny w swaggerze
    - photo.py - zawiera endpointy pobierające zdjęcia dla zalogowanych użytkowników oraz administratorów, oraz endpointy odpowiedzialne za operacje CRUD na zdjęciach
    - models.py - modele danych dla endpointów
- voivodeships/
    - public.py - plik zawierający endpoint do pobierania województw oraz szczegóły województw wraz z miastami jest klasyfikowany jako public api oraz widoczny w swaggerze
    - models.py - modele danych dla endpointów
- cities/
    - city.py - zawiera endpointy zarządzające miastami
    - models.py - modele danych dla endpointów
- tags/
    - public.py - plik zawierający endpoint do pobierania tagów jest klasyfikowany jako public api oraz widoczny w swaggerze
    - models.py - modele danych dla endpointów
- helpers/
    - db.py - dependency injection połączenia sql
    - models.py - wspólny schemat odpowiedzi błędów wykorzystywany w dokumentacji swagger

## 3.5 Uzasadnienie przyjętego rozwiązania aplikacja backendowa

Podział kodu oraz wykorzystane technologie w aplikacji backendowej zostały przyjęte z następujących powodów:

- FastApi - został wybrany ze względu na automatyczne generowanie interaktywnej dokumentacji Swagger dostępnej pod ścieżką /docs przy pomocy, której zostały udostępnione publiczne endpointy. Framework wspiera podział

kodu na niezależne moduły dzięki APIRouter, co bezpośrednio przekłada się na przyjętą strukturę folderów routes/.

- Podział na moduły per zasób - każdy zasób w systemie (auth, photos, voivodeships, cities, tags) posiada własny folder z plikiem zawierającym modele w pliku models.py, logikę endpointów publicznych w plikach public.py oraz logikę endpointów prywatnych w plikach z rozszerzeniem .py o nazwie folderu. Takie podejście zapewnia czytelność kodu, modyfikacja logiki zdjęć nie wymaga przeglądania pliku zawierającego zbędną logikę na przykład autoryzacji. Folder helpers/ przechowuje kod współdzielony między modułami odpowiadający na przykład za połączenie z bazą danych.
- Surowe zapytania sql w kodzie - bezpośredni SQL daje pełną kontrolę nad zapytaniem i pozwala precyzyjnie zoptymalizować złączenia tabel. Wszystkie parametry przekazywane są przez ? placeholdery, co chroni przed SQL Injection.
- Logowanie Google OAuth2 z JWT w ciasteczku – integracja z logowaniem dostarczonym przez google eliminuje potrzebę zarządzania hasłami. Wystawiony po zalogowaniu token JWT przechowywany jest w ciasteczku z flagą HttpOnly, pozwala to zapobiec atakom takim jak XSS. Weryfikacja tożsamości użytkownika przy każdym żądaniu nie wymaga zapytania do bazy danych, ponieważ wszystkie potrzebne dane są zakodowane w tokenie.
- Zmienne środowiskowe bez fallbacków do wartości domyślnych - wymusza jawną konfigurację środowiska i eliminuje ryzyko przypadkowego uruchomienia aplikacji z nieprawidłowymi domyślnymi wartościami

## 3.6 Struktura plików w aplikacji frontendowej

Aplikacja frontendowa w celu zapewnienia czytelności kodu została podzielona na foldery w obrębie głównego folderu src:

1.api – folder zawierający wspólne api dla całej aplikacji
2. components – folder podzielony jest na podfoldery definiujące różne komponenty aplikacji:

- AddEditModal – komponent przedstawiający modal odpowiedzialny za dodawanie oraz edycje zdjęć
- AdminLatestPhotoArchive - komponent zawierający widok archiwum dla administratora umożliwiające obsługę zdjęć innych użytkowników
- AdminPanel– komponent zawierający widok panelu administratora odpowiedzialnego za zarządzanie użytkownikami oraz edycją nazw dodanych miast
- Archive – komponent zawierający widok archiwum dostępnego dla każdego użytkownika
- AuthCallback – komponent, na który przekierowany jest użytkownik po zalogowaniu do aplikacji

- AuthGuard - komponent odpowiedzialny za zabezpieczenie nieuprawnionym osobą dostępu do ścieżek
- BanUserModal – komponent zawierający modal odpowiedzialny za obsługę banowania i odbanowania użytkowników
- DeletePhotoModal – komponent zawierający modal odpowiedzialny za usuwanie zdjęć
- ImageTile – generyczny komponent reprezentujący box w którym wyświetlane jest zdjęcie
- LoggedUserFilters – komponent zawierający menu z filtrami dla zalogowanego użytkownika zarówno administratora jak i zwykłego użytkownika
- Menu – komponent zawierający menu aplikacji
- MyArchive – komponent zawierający prywatne archiwum użytkownika zalogowanego
- Page – komponent zawierający wrapper na strone aplikacji
- PhotoGrid – komponent zawierający grid na bazie którego wyświetlane są zdjęcia wspólnie w archiwum
- ToastContainer – komponent zawierający obsługę toastów, za pomocą których odbywa się komunikacja z użytkownikiem
- VoivodeshipList – komponent zawierający listę województw wyświetlaną dla użytkownika

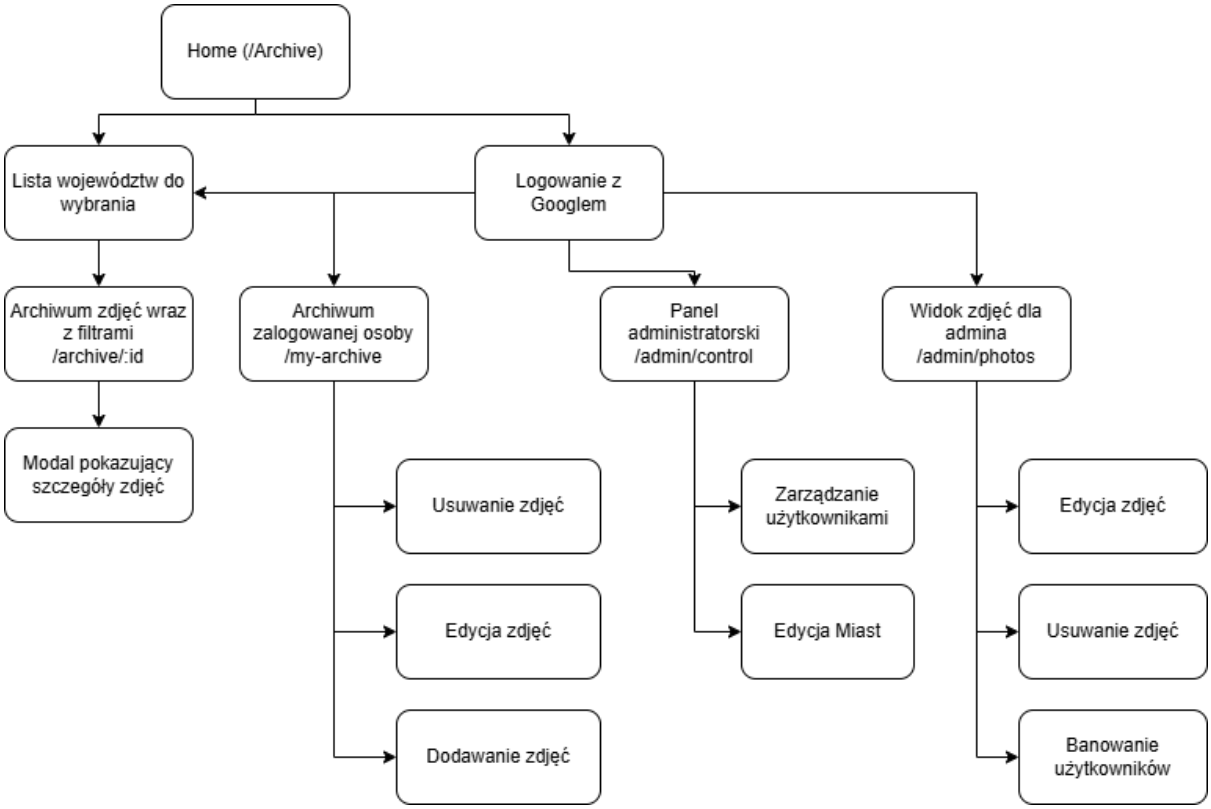
3. Contexts– zawiera kod kontekstu przechowującego dane zalogowanego użytkownika w całej aplikacji oraz kod kontextu odpowiedzialnego za obsługę wybierania motywu ciemnego, jasnego oraz wysoki kontrast

Dodatkowo do tego w głównym folderze aplikacji frontendowej znajduje się plik App.tsx odpowiedzialny za wystawienie ścieżek pod którymi znajdują się widoki aplikacji.

## 3.7 Architektura informacji

### 3.7.1 Nawigacja

Architektura informacji w systemie została przedstawiona na diagramie poniżej opisującym strukturę nawigacji aplikacji oraz dostęp do funkcjonalności w zależności od roli użytkownika.



Rysunek 10: Diagram nawigacji

Diagram nawigacji przedstawia strukturę dostępu do funkcjonalności systemu w zależności od roli użytkownika. Nawigacja w systemie dla niezalogowanego użytkownika odbywa się na podstawie hierarchii geograficznej. Użytkownikowi najpierw pokazuje się lista województw z której po wybraniu następuje przekierowanie na listę zdjęć dla danego województwa. Jest to hierarchia odzwierciedlająca naturalny podział występujący na mapach Polski.

Dla użytkownika zalogowanego nawigacja w archiwum pozostaje taka sama, natomiast dostępne są dodatkowe trzy sekcje:

- archiwum prywatne dla każdego zalogowanego użytkownika
- Archiwum administratorskie dla użytkowników z rolą administratora
- Panel administratorski dla użytkowników z rolą administratora

W archiwum prywatnym oraz w archiwum administratorskim pominięta jest ścieżka wybierania województwa, ponieważ przechowują one zdjęcia albo przypisane do użytkownika albo zdjęcia nowo dodane. Warto wspomnieć, że w obu tych archiwach zachowana jest hierarchia geograficzna poprzez udostępnienie filtru w panelu bocznym który umożliwia filtrowanie po województwach.

### 3.7.2 Filtry

W systemie zastosowano wyszukiwanie oparte na metadanych. Takie podejście umożliwia:

- szybkie odnalezienie informacji bez konieczności przeglądania całej struktury,
- elastyczne zawężanie wyników poprzez łączenie wielu kryteriów,
- zwiększenie trafności wyników dzięki zastosowaniu filtrów.

W widoku głównego archiwum dostępnego dla niezalogowanego oraz zalogowanego użytkownika dostępne są w menu widoku znajdującego się z boku ekranu następujące filtry pozwalające zawęzić obszar wyszukiwania:

- Przeszukiwanie w opisie – pole tekstowe które umożliwia wyszukiwanie danej frazy w opisie zdjęcia
- Miasta – selektor umożliwiający zawęzić wyszukiwane wyniki po miastach w województwie
- Daty – dwa pola wyboru daty odpowiedzialne za przedział czasowy w którym zostało zrobione zdjęcie.
- Tagi - możliwe wybranie zainteresowanych tagów z listy

W widoku prywatnym archiwum użytkownika zalogowanego dostępne są w menu widoku znajdującego się z boku ekranu następujące filtry pozwalające zawęzić obszar wyszukiwania:

- Województwa – selektor umożliwiający zawęzić wyszukiwane wyniki po województwie
- Miasta – selektor umożliwiający zawęzić wyszukiwane wyniki po miastach w województwie
- Daty – dwa pola wyboru daty odpowiedzialne za przedział czasowy w którym zostało zrobione zdjęcie.
- Tagi - możliwe wybranie zainteresowanych tagów z listy

W widoku archiwum adminowskim dostępne są w menu widoku znajdującego się z boku ekranu następujące filtry pozwalające zawęzić obszar wyszukiwania:

- Użytkownicy– selektor umożliwiający zawęzić wyszukiwane wyniki po
- użytkowniku
- Województwa – selektor umożliwiający zawęzić wyszukiwane wyniki po województwie
- Miasta – selektor umożliwiający zawęzić wyszukiwane wyniki po miastach w województwie
- Daty – dwa pola wyboru daty odpowiedzialne za przedział czasowy w którym zostało zrobione zdjęcie.
- Tagi - możliwe wybranie zainteresowanych tagów z listy
- Checkbox od ostatniego logowania – checkbox zawężający zdjęcia do czasu ostatniego logowania.

### 3.7.3 Metadane

Każde zdjęcie w systemie opisane jest następującymi metadanymi:

- Opis - pole tekstowe wymagane przy dodawaniu zdjęcia, umożliwia wyszukiwanie pełno tekstowe po frazie.
- Lokalizacja - składa się z dwóch poziomów: województwa oraz miasta. Miasto pochodzi ze słownika lub może zostać dodane przez użytkownika jako nowe przy wgrywaniu zdjęcia. Województwo jest dostępne z słownika województw.
- Data wykonania - przechowywana w trzech osobnych polach: rok, miesiąc, dzień, pozwala to na zapisanie daty z dowolną precyzją samo 1965, gdy znany jest tylko rok, 06.1965, gdy znany jest miesiąc lub pełna data.
- Tagi – od 0 do 5 tagów wybieranych z predefiniowanego słownika. Zamknięty słownik eliminuje duplikaty i literówki. Wyszukiwanie po tagach działa jako logiczne AND zdjęcie musi posiadać wszystkie wybrane tagi jednocześnie.
- Plik – przechowywany w MinIO pod losową nazwą UUID z oryginalnym rozszerzeniem.

### 3.7.4 Uzasadnienie Architektury Informacji

W projekcie zastosowano strukturę opartą na hierarchii geograficznej województwo => miasto => zdjęcia, która odzwierciedla naturalny sposób postrzegania informacji przez użytkowników po lokalizacji geograficznej w Polsce. Taki model organizacji treści jest intuicyjny, ponieważ użytkownicy najczęściej kojarzą fotografie historyczne z konkretną lokalizacją. Zastosowanie tej struktury rozwiązuje następujące problemy:

- Ułatwia orientację w systemie i szybkie zawężanie obszaru wyszukiwania
- Redukuje liczbę kroków potrzebnych do znalezienia zdjęć
- Zapewnia spójność między różnymi widokami systemu

Dodatkowo zastosowano do szczegółowego wyszukiwania udostępnione zostały filtry. Pozwalają one uniknąć nadmiernej głębokości struktury oraz umożliwiają użytkownikowi elastyczne przeszukiwanie danych bez konieczności przechodzenia przez wiele poziomów nawigacji.

## 3.8 Uzasadnienie przyjętego rozwiązania aplikacja frontendowa

Podział kodu, widoków oraz wykorzystane technologie w aplikacji frontendowej zostały przyjęte z następujących powodów:

- Biblioteka Material UI - gotowy zestaw komponentów zgodnych z wytycznymi dostępności WCAG z pudełka. Wbudowany system theming przez ThemeProvider pozwala obsłużyć trzy motywy (jasny, ciemny, wysoki kontrast)

bez duplikowania styli w każdym komponencie wystarczy jedna zmiana w ThemeContext aby zmienić wygląd całej aplikacji.

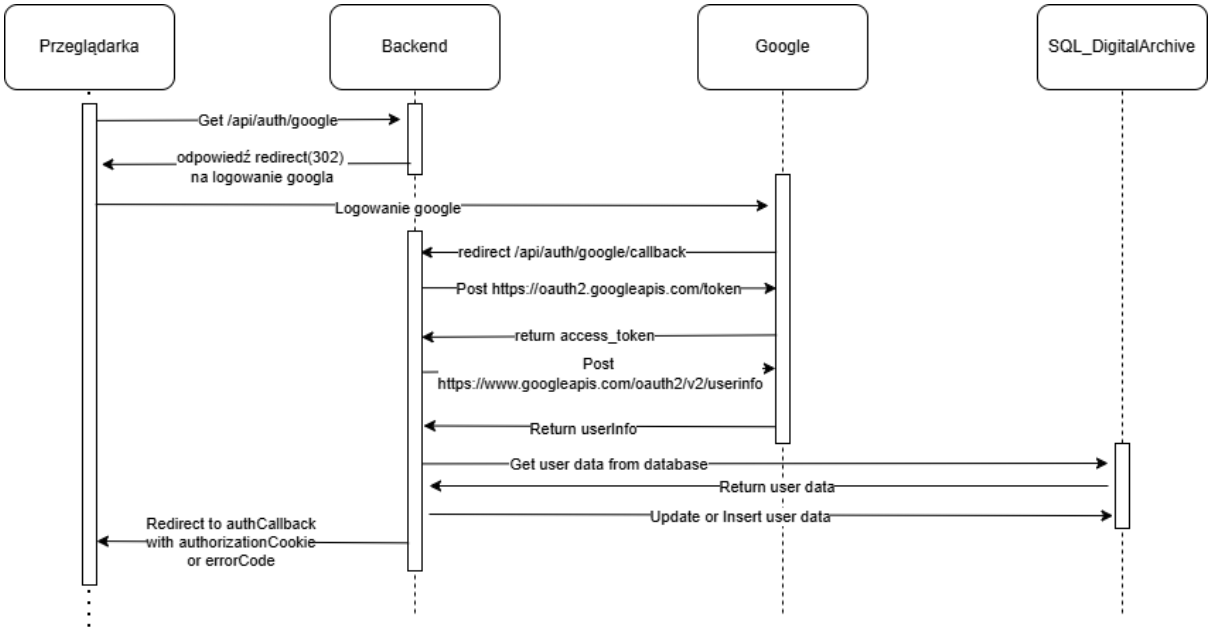
- Axios - axios skonfigurowany z withCredentials: true automatycznie dołącza ciasteczko sesji do każdego żądania backendowego
- React Context – wykorzystanie z kontekstu w celach przechowywania danych autoryzacyjnych oraz theme powoduje, że nie trzeba tych danych przekazywać w propsach tylko są dostępne w obrębie ContextProviderów dla każdego komponentu
- React z Typescript – react jest najpopularniejszym frameworkiem do budowania interfejsów użytkownika. Typescript pozwala na dbanie o poprawne typy danych
- Podział komponentów na foldery - każda funkcjonalność systemu posiada własny folder w Components/ zawierający wszystkie pliki związane z danym komponentem. Modyfikacja widoku archiwum nie wymaga przeglądania plików związanych z innym widokiem na przykład panelem administratora.

## 3.9 Bezpieczeństwo

- Logowanie przez google eliminuje potrzebe zarządzania hasłami
- Token w ciasteczku httponly zapobiega kodu javascript dostępu do ciasteczka
- Parametryzowane zapytania SQL poprzez ? zapobiegają atakom takim jak SQL injection
- Przy każdym zapytaniu wymagającym odpowiednej roli użytkownika pobierana jest wartość z ciasteczka oraz weryfikacja czy użytkownik ma odpowiednią rolę po stronie aplikacji backendowej
- AuthGuard – komponent weryfikujący rolę użytkownika chroniący nieuprawnione osoby przed dostępem do danej ścieżki

## 3.10 Autoryzacja

Autoryzacja w systemie zintegrowana jest z logowaniem dostarczonym przez firmę Google. Poniższy diagram prezentuje flow autoryzacyjne:




Rysunek 11: Diagram autoryzacyjny

# 4. Dostępność w aplikacji

Rozdział zawiera podsumowanie dostępności w aplikacji. System został zaprojektowany w taki sposób, aby był zgodny z wytycznymi WCAG 2.1 na poziomie AA. Warto wspomnieć, że każdy z widoków został zweryfikowany z wtyczką WAVE w Google Chrome która nie wykazywała błędów ani ostrzeżeń. Zastosowane rozwiązania obejmują.

- Alternatywne teksty dla zdjęć
- Zaimplementowane trzy motywy kolorystyczne zapewniające odpowiedni kontrast tekstu:
    - Motyw jasny - domyślny motyw
    - Motyw Ciemny
    - Wysoki kontrast
- Wszystkie wykorzystane elementy select oraz pola tekstowe zawierają label poprawnie opisujące cel pola
- Komponenty MUI obsługują nawigację klawiaturą z pudełka: Tab do przechodzenia między elementami, Enter i Spacja do aktywacji, strzałki w komponentach Select i Autocomplete.
- Wykorzystany atrybut lang w elemencie html wskazujący na język polski dla czytników
- Spójny i przewidywalny interfejs przykładem może być komponent Menu, który jest renderowany w sposób spójny między widokami

- Każde błędy walidacyjne oraz api są komunikowane przez Toast z rolą alert przez co czytniki ekranu od razu komunikują błąd. System komunikuje każdy błąd nie ma pominiętych błędów
- Wykorzystanie z atrybutów aria w implementacji w celu popprawy dostępności dla czytników ekranów:
    - Aria-label opisujące mniej zrozumiałe elementy jak wybierak daty od lub daty do
    - Klikalne kafelki zdjęć mają rolę button
    - Modale mają w mui wbudowaną rolę dialog

Zastosowane rozwiązania spełniają główne zasady WCAG:

- Perceivable (postrzegalność),
- Operable (funkcjonalność),
- Understandable (zrozumiałość),
- Robust (solidność).

# 5. Endpointy

Endpointy udostępnione w aplikacji można rozdzielić na dwa typy: prywatne oraz publiczne. Publiczne api zostało udostępnione w postaci dokumentacji swagger pod ścieżką {backend_endpoint_url}/docs. Lista endpointów:

**1. Prywatne:**

- GET /auth/google - autoryzacja google
  Request: -
  Response: redirect to accounts.google.com

- GET /auth/google/callback - callback autoryzacyjny
  Request: -
  Response: redirect to fe /auth/callback

- POST /auth/logout - wylogowanie
  Request: -
  Response: string

- GET /auth/me - pobieranie danych zalogowanej osoby
  Request: -
  Response:
  {
    "id": int,

"sub": string,
"email": string,
"name": string,
"roles": lista stringów
}

- GET /auth/admin/users - pobieranie listy użytkowników
  Request: -
  Response: [
  {
  "id": int,
  "email": string,
  "displayName": string,
  "isActive": bool,
  "photoCount": int
  }
  ]

- POST /auth/admin/users/{user_id}/ban - zablokowanie użytkownika
  Request: -
  Response: string

- POST /auth/admin/users/{user_id}/unban - odblokowanie użytkownika
  Request: -
  Response: string

- GET /photos/logged - pobieranie zdjęć zalogowanego użytkownika
  Request: parametry query:

  voivodeshipId: int (opcjonalny)
  cityId: int (opcjonalny)
  dateFrom: string YYYY-MM-DD (opcjonalny)
  dateTo: string YYYY-MM-DD (opcjonalny)
  tags: lista stringów (opcjonalna)
  page: int (opcjonalny, domyślnie 1)
  pageSize: int (opcjonalny, domyślnie 10, max 50)

  Response: {
  "total": int,
  "items": [

```
    {
      "id": int,
      "cityId": int,
      "cityName": string,
      "voivodeshipName": string,
      "description": string,
      "storedFileName": string,
      "takenYear": int lub null,
      "takenMonth": int lub null,
      "takenDay": int lub null,
      "tags": lista stringów
    }
  ]
}
```

- GET /photos/admin - pobieranie wszystkich zdjęć dla administratora
  Request: parametry query:

  userId: int (opcjonalny)
  voivodeshipId: int (opcjonalny)
  cityId: int (opcjonalny)
  dateFrom: string YYYY-MM-DD (opcjonalny)
  dateTo: string YYYY-MM-DD (opcjonalny)
  tags: lista stringów (opcjonalna)
  isSinceLastLoginChecked: bool (opcjonalny)
  page: int (opcjonalny, domyślnie 1)
  pageSize: int (opcjonalny, domyślnie 10, max 50)

  Response: {

```
  "total": int,
  "items": [
    {
      "id": int,
      "cityId": int,
      "cityName": string,
      "voivodeshipName": string,
      "description": string,
      "storedFileName": string,
      "takenYear": int lub null,
      "takenMonth": int lub null,
      "takenDay": int lub null,
```

"tags": lista stringów,
"createdByUserId": int,
"createdByDisplayName": string,
"createdByIsActive": bool
}
]
}

- POST /photos - dodanie zdjęcia
  Request: {
  "file": string (base64 data URL),
  "fileName": string,
  "description": string,
  "cityId": int,
  "customCityName": string (opcjonalny),
  "voivodeshipId": int (opcjonalny),
  "takenYear": int lub null,
  "takenMonth": int lub null,
  "takenDay": int lub null,
  "tags": lista stringów (opcjonalna, max 5)
  }

  Response: string

- PATCH /photos/{id} - aktualizacja zdjęcia
  Request: {
  "description": string,
  "cityId": int,
  "customCityName": string (opcjonalny),
  "voivodeshipId": int (opcjonalny),
  "takenYear": int lub null,
  "takenMonth": int lub null,
  "takenDay": int lub null,
  "tags": lista stringów lub null,
  "file": string lub null (opcjonalny, base64 data URL),
  "fileName": string (opcjonalny)
  }

  Response: string

- DELETE /photos/{id} - usunięcie zdjęcia
  Request: -
  Response: string

- GET /cities - pobieranie miast dla województwa
  Request: parametry query: voivodeshipId: int
  Response: [
  {
  "id": int,
  "name": string
  }
  ]

- GET /cities/all - pobieranie wszystkich miast
  Request: -
  Response: [
  {
  "id": int,
  "name": string,
  "voivodeshipName": string
  }
  ]

- PATCH /cities/{city_id} - zmiana nazwy miasta
  Request: {
  "name": string
  }

  Response: string

**2. Publiczne:**

- GET /photos - pobieranie zdjęć
  Request parametry query:

  voivodeshipId: int (opcjonalny)
  cityId: int (opcjonalny)
  text: string (opcjonalny)
  dateFrom: string YYYY-MM-DD (opcjonalny)
  dateTo: string YYYY-MM-DD (opcjonalny)

tags: lista stringów (opcjonalna)
page: int (opcjonalny, domyślnie 1)
pageSize: int (opcjonalny, domyślnie 10, max 50)

Response: {
 "total": int,
 "items": [
  {
   "id": int,
   "cityId": int,
   "cityName": string,
   "voivodeshipName": string,
   "description": string,
   "storedFileName": string,
   "takenYear": int lub null,
   "takenMonth": int lub null,
   "takenDay": int lub null,
   "tags": lista stringów
  }
 ]
}

- GET /voivodeships - pobieranie listy województw
  Request: -
  Response: [
   {
    "id": int,
    "name": string
   }
  ]

- GET /voivodeships/details - pobieranie szczegółów województwa z miastami
  Request parametry query: Id: int
  Response: {
   "voivodeship": {
    "id": int,
    "name": string
   },
   "cities": [
    {
     "id": int,

```
      "name": string
    }
  ]
}
```

- GET /tags - pobieranie listy tagów
  Request: -
  Response:
  ```
  [
   {
     "id": int,
     "name": string
   }
  ]
  ```